

SRI Tourism: Sistema de Recuperación de Información para Turismo

Equipo de desarrollo¹

Facultad de Matemática y Computación, Universidad de La Habana, La Habana,
Cuba

<https://github.com/Ronald1301/Project-SRI-Tourism>

Resumen SRI Tourism es un sistema de recuperación de información orientado al dominio turístico, diseñado para recopilar, normalizar, indexar y consultar documentos procedentes de la web y de un corpus local curado. El sistema integra un crawler modular por sitios, filtros de calidad lingüística y textual, una base de datos vectorial con Sentence Transformers y FAISS HNSW, índices clásicos TF-IDF y LSI, y una capa RAG con respuesta generada por un modelo local. La arquitectura fue pensada para sostener búsquedas rápidas, persistencia estable de vectores, ampliación dinámica del corpus mediante búsqueda web y una interfaz visual simple, legible y centrada en la consulta del usuario. Este informe documenta el diseño técnico, el uso del sistema, la organización visual de la interfaz y una valoración crítica de sus fortalezas y limitaciones.

Keywords: Recuperación de información · Turismo · RAG · FAISS · Sentence Transformers · Crawling web

1. Introducción

El proyecto SRI Tourism resuelve un problema clásico de recuperación de información en un dominio específico: encontrar documentos relevantes sobre turismo en Cuba, rankearlos de forma consistente y, cuando la evidencia disponible es insuficiente, ampliar la búsqueda hacia la web. El sistema combina recuperación lexical, recuperación semántica y generación de respuesta, lo que permite cubrir consultas descriptivas, geográficas y comparativas con una única interfaz.

La decisión de separar el sistema en módulos independientes responde a dos objetivos. Primero, facilitar la evolución del proyecto sin acoplar la ingesta de datos, el indexado y la respuesta generada. Segundo, hacer posible el reemplazo o ajuste de cada estrategia de búsqueda sin afectar el resto del flujo. Esa modularidad también simplifica la evaluación, porque cada componente puede probarse de forma aislada.

2. Documentación técnica del sistema

2.1. Arquitectura general

La arquitectura se organiza en cinco bloques:

- **Adquisición de datos:** crawler especializado por sitios y módulo de búsqueda web.
- **Preprocesamiento:** limpieza, tokenización, stopwords y stemming.
- **Indexación:** TF-IDF, LSI y base de datos vectorial con embeddings.
- **Recuperación y generación:** búsqueda vectorial, LSI, híbrida y RAG.
- **Presentación:** API FastAPI y frontend con Flet.

El flujo principal es: documentos crudos → preprocesamiento → índices → búsqueda → respuesta final. Cuando la evidencia local no alcanza un umbral de suficiencia, el sistema activa el módulo de búsqueda web, incorpora nuevos documentos y vuelve a consultar.

2.2. Ingesta de datos y crawling

La arquitectura de crawling se organiza en dos fases complementarias. La **fase inicial** construye el corpus local a partir de fuentes turísticas seleccionadas por su cercanía al dominio del problema. En el proyecto se priorizaron portales como *Visitar Cuba*, *Cuba Travel* e *Infotur*, ya que concentran información directamente relacionada con destinos, servicios, atractivos, alojamiento y orientación turística sobre Cuba. Estas fuentes representan bien el dominio porque publican contenido especializado, estable en el tiempo y orientado a usuarios que buscan información turística estructurada. El objetivo de esta etapa es capturar documentos relevantes y reutilizables bajo restricciones estrictas de dominio, esquema, patrones de URL y respeto a `robots.txt`. En esta etapa se prioriza la calidad del corpus sobre la cobertura exhaustiva, evitando descargas redundantes y recursos no textuales.

Antes de almacenar un documento, el sistema aplica filtros de calidad que reducen ruido y estabilizan la colección:

- idioma detectado en español o inglés;
- mínimo de palabras para descartar fragmentos triviales;
- exclusión de páginas con bajo contenido informativo;
- descarte de enlaces redundantes, externos o fuera del objetivo temático.

La **fase dinámica** actúa como mecanismo de ampliación bajo demanda. Se activa cuando la evidencia local no es suficiente para responder una consulta con la calidad esperada. En esa etapa, el sistema utiliza *DuckDuckGo* como motor de descubrimiento web para localizar páginas potencialmente útiles fuera del corpus local. A partir de esos resultados, evalúa la relevancia de las URLs antes de descargarlas, sigue enlaces solo hasta una profundidad limitada, respeta `robots.txt` y aplica timeouts para evitar sobrecarga en los sitios consultados.

En esta fase dinámica, la selección no depende únicamente de la coincidencia léxica. Primero se filtran URLs por importancia temática y luego se evalúa la relevancia semántica del contenido recuperado frente al dominio turístico y frente a la consulta concreta del usuario. De este modo, el sistema no incorpora cualquier resultado web, sino solo aquel que aporta evidencia consistente y útil para la recuperación posterior.

2.3. Preprocesamiento

El preprocesamiento actúa como la capa de normalización lingüística del sistema. Su función es transformar texto bruto en representaciones consistentes, comparables y adecuadas para recuperación de información. Para ello, reduce ruido superficial, elimina elementos poco informativos y homogeneiza variaciones ortográficas que no aportan significado semántico.

En términos técnicos, esta capa combina limpieza textual, segmentación léxica, filtrado de palabras vacías y stemming. Con ello se logra disminuir la dispersión del vocabulario, mejorar el solapamiento entre consultas y documentos, y estabilizar tanto los índices léxicos como los vectores semánticos. El mismo criterio se aplica a documentos del corpus local y a textos recuperados desde la web, de modo que ambas fuentes entren al sistema con una normalización homogénea.

Además, la capa de ingesta soporta formatos heterogéneos de entrada. Esto permite incorporar contenido estructurado y semiestructurado sin exigir conversiones manuales previas. El tratamiento por documento independiente preserva la granularidad documental del corpus y facilita el posterior indexado incremental.

2.4. Índices clásicos: TF-IDF y LSI

La capa clásica de recuperación combina un modelo TF-IDF con una proyección semántica mediante LSI. TF-IDF cumple dos funciones: conserva una representación léxica interpretable del corpus y proporciona la matriz base sobre la que se calcula el espacio latente. LSI reduce esa matriz a una dimensionalidad menor, agrupando términos y documentos que coaparecen en contextos similares.

Esta estrategia es útil cuando la consulta expresa la misma intención con un vocabulario distinto al del documento. Frente a una coincidencia puramente léxica, LSI suaviza la rigidez de la comparación término a término y recupera relaciones semánticas aproximadas. El resultado es una capa clásica más robusta que un simple conteo de palabras, aunque todavía interpretable y eficiente para el dominio del proyecto.

En el sistema, esta familia de índices funciona como una base de referencia rápida para consultas donde la recuperación vectorial no sea la mejor opción o cuando se busque complementar señales distintas mediante búsqueda híbrida.

2.5. Modelo de semántica latente

El modelo de semántica latente, también conocido como LSI (*Latent Semantic Indexing*), es una técnica que proyecta documentos y consultas a un espacio latente de menor dimensión para capturar relaciones semánticas implícitas entre términos. Su valor principal no está en contar palabras, sino en descubrir estructuras ocultas de coocurrencia dentro del corpus. De esta manera, términos distintos que aparecen en contextos similares pueden quedar próximos en el espacio latente, lo que ayuda a resolver problemas de sinonimia parcial y variación lexical.

En un sistema turístico como este, LSI es útil porque una misma intención de búsqueda puede expresarse con vocabulario distinto según la fuente o el usuario. Por ejemplo, una consulta sobre “playas” puede relacionarse con documentos que mencionan “balnearios”, “costa” o “zonas de baño” sin que exista una coincidencia exacta de palabras. Esa capacidad de aproximación semántica hace que LSI sea un complemento valioso frente a la búsqueda puramente léxica, especialmente cuando el corpus todavía es limitado o la consulta es breve.

2.6. Base de datos vectorial

La base de datos vectorial organiza la recuperación semántica mediante embeddings generados con modelos de tipo *Transformer*. Se eligió esta familia de modelos porque representa el significado de un texto a partir de su contexto completo, no solo de la coincidencia aislada de palabras. Esa propiedad es especialmente valiosa en recuperación de información, ya que permite reconocer consultas que expresan la misma intención con formulaciones distintas.

En lugar de depender únicamente de coincidencias léxicas, los Transformers producen vectores densos capaces de capturar relaciones semánticas entre frases, títulos y fragmentos documentales. Para el dominio turístico esto resulta muy útil, porque una misma idea puede expresarse de muchas maneras: nombres de lugares, servicios, actividades, recomendaciones o expresiones coloquiales.

El modelo escogido fue **paraphrase-multilingual-MiniLM-L12-v2**. Su selección responde a varios criterios técnicos. Primero, es multilingüe, por lo que puede trabajar de forma consistente con contenido en español e inglés, que son los idiomas prioritarios del sistema. Segundo, ofrece un buen equilibrio entre calidad semántica y costo computacional, lo que lo hace adecuado para un flujo interactivo donde la latencia importa. Tercero, está diseñado para tareas de similitud semántica y recuperación de frases o fragmentos, exactamente el tipo de problema que resuelve esta base vectorial.

El diseño parte de una observación importante: un documento largo puede contener varios subtemas, por lo que asignarle un único vector suele mezclar señales distintas y reducir la calidad de la búsqueda. Para corregir eso, el sistema segmenta cada documento en fragmentos semánticos y genera una representación vectorial por fragmento.

Los vectores se almacenan en formato de precisión simple, una decisión habitual en sistemas de recuperación semántica porque ofrece una relación equilibrada entre costo de almacenamiento, velocidad de cálculo y robustez numérica.

Además, la carga diferida de artefactos grandes permite tratar colecciones más voluminosas sin exigir que todo el conjunto permanezca íntegramente copiado en memoria desde el inicio.

La indexación se apoya en FAISS con un índice HNSW. Esta estructura es adecuada para recuperación aproximada de vecinos más cercanos porque organiza los vectores en un grafo de proximidad multinivel. En lugar de comparar la consulta con todos los vectores del corpus, la búsqueda navega regiones prometedoras del espacio vectorial y reduce de forma significativa la latencia en consultas interactivas.

El método de búsqueda utilizado es, por tanto, una *Approximate Nearest Neighbor Search* (ANN) sobre vectores densos normalizados. La consulta se transforma al mismo espacio semántico que los documentos y se evalúa contra el índice para recuperar los fragmentos más próximos en términos de similitud. Cuando los vectores están normalizados, esta proximidad se interpreta como una aproximación eficiente a la similitud coseno. El resultado no es una coincidencia exacta de palabras, sino una medida de cercanía semántica entre la intención de la consulta y los fragmentos del corpus.

Ese beneficio viene acompañado de un costo técnico razonable:

- mayor consumo de memoria por la estructura del grafo;
- construcción más costosa;
- resultados aproximados, no exactos.

Ese costo es aceptable porque el sistema prioriza latencia de respuesta y escalabilidad para consultas frecuentes. La persistencia del índice y de los vectores permite reiniciar el sistema sin reconstruir completamente la base, y la actualización incremental evita rehacer el trabajo ya indexado.

La organización de metadatos también se rediseñó para separar el identificador documental del índice posicional interno. Esto mejora la trazabilidad, simplifica la actualización incremental y hace más robusta la relación entre cada vector, su documento de origen y la información auxiliar asociada.

2.7. RAG

El componente RAG combina recuperación y generación en una sola tubería. En esta versión, la recuperación se basa únicamente en una estrategia híbrida que fusiona una señal léxica con una señal semántica. La señal léxica aporta coincidencia terminológica y la señal semántica aporta generalización de significado, lo que permite recuperar evidencia más robusta en consultas turísticas heterogéneas.

La estrategia híbrida resulta especialmente útil cuando una consulta mezcla vocabulario específico con intención semántica general. En un dominio turístico esto ocurre con frecuencia, ya que un mismo concepto puede expresarse con nombres de lugares, tipos de servicios, actividades o descripciones informales.

Una vez obtenida la evidencia, el sistema estima si el conjunto recuperado es suficiente. Si lo es, la respuesta se genera directamente a partir del contexto local.

Si no lo es, se activa la ampliación dinámica con búsqueda web, se incorporan nuevos documentos al flujo de recuperación y se vuelve a consultar antes de producir la respuesta final. La generación final se apoya en un modelo local para mantener independencia operativa y reproducibilidad.

Recuperación híbrida La recuperación híbrida combina dos familias de señales complementarias: una señal léxica basada en TF-IDF y LSI, y una señal semántica basada en embeddings densos. La motivación principal es que ambas estrategias capturan aspectos distintos de una consulta. La recuperación léxica es fuerte cuando existe coincidencia terminológica clara, mientras que la recuperación semántica resulta superior cuando la misma intención se expresa con vocabulario diferente.

En la parte vectorial, la similitud entre una consulta q y un documento d puede expresarse como producto interno entre vectores normalizados:

$$\text{sim}_{\text{vec}}(q, d) = \hat{q} \cdot \hat{d}$$

Si $\hat{q}$ y $\hat{d}$ están normalizados, esta medida coincide con una aproximación a la similitud coseno. En la parte léxica, LSI proyecta los textos a un espacio latente de menor dimensión antes de ordenar los resultados, lo que permite capturar relaciones semánticas implícitas entre términos.

La fusión híbrida permite aprovechar lo mejor de ambos enfoques en una sola respuesta ordenada. En lugar de elegir una sola representación, el sistema integra múltiples rankings y conserva los documentos que aparecen con buena posición relativa en más de una fuente. Esto reduce el riesgo de perder resultados útiles por depender de un único criterio de similitud.

La fusión de rankings se formula mediante Reciprocal Rank Fusion (RRF). Para un documento d , si $r_i(d)$ denota su posición en la lista ordenada producida por el recuperador i , entonces el puntaje combinado se define como:

$$\text{RRF}(d) = \sum_{i=1}^m \frac{1}{k + r_i(d)}$$

donde m es el número de recuperadores combinados y k es una constante de suavizado que reduce el peso de posiciones muy altas. Este esquema favorece documentos que aparecen bien posicionados en varias listas, en lugar de depender de una sola señal.

En la práctica, este enfoque es especialmente valioso en un dominio turístico, donde los usuarios pueden formular consultas de muchas formas: nombres de lugares, tipos de servicios, actividades, sinónimos o descripciones informales. La recuperación híbrida mejora la cobertura sin sacrificar por completo la precisión, y actúa como una solución intermedia entre la exactitud léxica y la generalización semántica.

2.8. API y frontend

La capa de servicio expone la funcionalidad del sistema como una interfaz HTTP apta para consumo externo. Su responsabilidad es coordinar la consulta,

el modo de recuperación y la cantidad de resultados, y devolver una respuesta coherente que combine evidencia recuperada y síntesis generada.

La interfaz visual actúa como una capa de interacción centrada en la consulta. Su diseño privilegia una experiencia de uso directa: el usuario formula la consulta, selecciona el modo de recuperación y recibe tanto la respuesta sintetizada como los resultados de respaldo. El objetivo no es ofrecer múltiples paneles de navegación, sino una vista única y clara para recuperar información con el menor esfuerzo cognitivo posible.

3. Manual de usuario

3.1. Requisitos mínimos

Para ejecutar el sistema se necesita:

- Python 3.12 o superior;
- dependencias instaladas con `pip install -r requirements.txt`;
- opcionalmente Docker y Docker Compose;
- Ollama instalado si se desea respuesta generada localmente.

3.2. Ejecución del sistema

Los comandos más útiles son:

```
python3 main.py crawl
python3 main.py vectordb
python3 main.py lsi_train
python3 main.py query "playas en cuba" --top-k 5
python3 main.py lsi_query "turismo en cuba" --top-k 5
python3 main.py rag_query "playas en cuba" --top-k 4
python3 main.py web_search "playas cubanas" --top-k 5
```

También puede ejecutarse la interfaz visual con:

```
python3 src/frontend/app.py
```

Si se usa Docker, el sistema puede correr en un entorno reproducible sin depender de la configuración local.

3.3. Cómo definir una consulta

El usuario debe escribir una consulta en lenguaje natural, por ejemplo:

- *playas tranquilas en Cuba*;
- *qué hacer en Trinidad en un fin de semana*;
- *hoteles con buena ubicación en La Habana*;
- *lugares turísticos en Baracoa*.

Luego envía la consulta y la cantidad de resultados. El sistema devuelve una lista rankeada de documentos y, si corresponde, una respuesta generada con apoyo del contexto recuperado.

3.4. Descripción de la interfaz visual

La interfaz está compuesta por tres zonas principales:

- **barra superior de consulta:** campo de texto y selector de `top_k`;
- **área de respuesta:** muestra la respuesta generada por RAG cuando existe;
- **lista de resultados:** tarjetas con título, fragmento, relevancia, fuente y metadatos.

Si existen más resultados disponibles, el sistema muestra el botón *Cargar más*. El diseño prioriza la consulta y la respuesta sobre la navegación secundaria.

4. Diseño de la interfaz visual

El diseño visual se construyó bajo una lógica de lectura rápida y baja carga cognitiva. La consulta se ubica en la parte superior porque es la acción primaria del usuario. Los resultados se presentan debajo en tarjetas independientes, lo que facilita escaneo, comparación y apertura de fuentes.

La tarjeta de resultado incluye primero la relevancia, luego el título, el fragmento y finalmente los metadatos. Ese orden no es casual: la relevancia ayuda a decidir si seguir leyendo, el título permite reconocer la fuente, el fragmento aporta contexto y los metadatos sirven como señales auxiliares. La URL se muestra al final porque es útil para verificación, pero no debe competir visualmente con el contenido principal.

La respuesta RAG se coloca antes de la lista de documentos porque actúa como síntesis primaria. De este modo, el usuario ve primero la respuesta y después las evidencias que la sustentan. Esta decisión reduce el esfuerzo de interpretación y hace que el sistema se comporte como un asistente, no solo como un buscador.

El uso de un tema oscuro responde a dos criterios: mejorar contraste en tarjetas y disminuir fatiga visual durante sesiones largas. La composición vertical con scroll favorece dispositivos con pantallas pequeñas y evita dividir demasiado la atención del usuario. En conjunto, el posicionamiento de la información sigue una estrategia de progresiva divulgación: primero la respuesta, luego los resultados, luego el acceso a la fuente.

5. Justificación de las estrategias de posicionamiento

Las estrategias de posicionamiento fueron elegidas para maximizar claridad y reducir fricción:

- **Consulta arriba:** el usuario no debe buscar dónde escribir.
- **Respuesta antes de documentos:** la síntesis debe aparecer antes que la evidencia bruta.
- **Tarjetas separadas:** cada documento se interpreta como unidad independiente.

- **Metadatos como etiquetas:** facilitan evaluación rápida sin saturar el texto principal.
- **Carga progresiva:** evita listas excesivamente largas en la primera interacción.

Estas decisiones son coherentes con el objetivo del sistema: apoyar recuperación de información y generación de respuestas, no solo listar documentos.

6. Opinión crítica

El proyecto tiene varias bondades técnicas. La principal es la modularidad: cada etapa del sistema puede evolucionar sin rehacer todo el flujo. También destaca la combinación de búsqueda lexical, semántica y generativa, que mejora la cobertura frente a consultas diversas. Otro punto fuerte es la persistencia correcta de datos e índices, incluyendo embeddings, FAISS, metadatos y documentos web.

La segunda fortaleza es el uso de filtros de calidad en la web. No se descarga ni se indexa todo indiscriminadamente; primero se aplica selección por URL, luego calidad textual, y después relevancia temática. Eso reduce ruido y mejora la utilidad del corpus ampliado.

Como insuficiencia, el sistema todavía depende de heurísticas para varias tareas delicadas: detección de idioma, ratio de boilerplate, densidad de enlaces y suficiencia de información. Son reglas útiles, pero pueden fallar en páginas con estructura atípica. Además, la evaluación automática aún no cubre todos los escenarios de uso real y el corpus sigue siendo relativamente pequeño frente al dominio turístico completo.

7. Deficiencias detectadas y posibles soluciones

- **Cobertura del corpus limitada:** el sistema depende mucho de las páginas iniciales y de la calidad del crawling. Solución: ampliar seeds, automatizar nuevas reglas por sitio y programar re-crawls periódicos.
- **Heurísticas frágiles:** algunos filtros de calidad pueden rechazar contenido válido. Solución: entrenar clasificadores ligeros de calidad documental o ajustar umbrales por dominio.
- **Evaluación todavía parcial:** falta una batería más amplia de consultas y juicios de relevancia. Solución: construir un conjunto de evaluación más grande y medir precisión, cobertura y latencia por módulo.
- **Interfaz funcional pero sobria:** la UI cumple su rol, pero puede enriquecerse. Solución: agregar resaltado de citas, filtros por fuente, historial y vista expandida de resultados.
- **Búsqueda web costosa en casos extremos:** aunque se limita por número de páginas, puede seguir siendo lenta en sitios complejos. Solución: incorporar colas asíncronas, cache de respuestas y priorización más agresiva.
- **Dependencia de Ollama para la generación:** si el modelo local no está disponible, la respuesta se degrada. Solución: ofrecer más de un backend generativo o una respuesta extractiva de respaldo mejorada.

8. Conclusiones

SRI Tourism demuestra que es posible construir un sistema de recuperación de información especializado, persistente y extensible con una arquitectura modular relativamente compacta. La combinación de crawling controlado, pre-procesamiento, TF-IDF, LSI, FAISS HNSW y RAG ofrece una base sólida para recuperar información turística de forma eficiente y generar respuestas útiles sobre esa evidencia.

El trabajo realizado deja una plataforma lista para evolucionar hacia una solución más robusta, con mejor evaluación, mejor cobertura del corpus y una interfaz más rica. Aun con sus limitaciones, el sistema ya cumple el objetivo principal del proyecto: buscar, ampliar y responder sobre información turística con una estructura técnica clara y mantenible.

Referencias

1. Lewis, P., Perez, E., Piktus, A., et al.: Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Advances in Neural Information Processing Systems (2020)
2. Reimers, N., Gurevych, I.: Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. Proceedings of EMNLP (2019)
3. Johnson, J., Douze, M., Jégou, H.: Billion-scale similarity search with GPUs. IEEE Transactions on Big Data (2019)
4. Cormack, G. V., Clarke, C. L. A., Büttcher, S.: Reciprocal rank fusion outperforms condorcet and individual rank learning methods. Proceedings of SIGIR (2009)
5. FAISS Documentation. <https://github.com/facebookresearch/faiss>